Z. 2026 **Concurrency Theory** P. Hofman

Tutorial 1, Friday, 02/10/2026, by VRM

Today: *Transition Systems & Operational Models*

Goal. This tutorial introduces a common way to describe the step-by-step behaviour of a system. We will turn operational rules into a labelled transition system (LTS), explore the reachable part of that system, and compare models with different kinds of configurations.

A small amount of theory

Labelled transition systems. A labelled transition system is a tuple

$$\mathcal{T} = (C, \mathsf{Act}, \Delta, c_0),$$

where C is a set of states (also called configurations), Act is a set of action labels, $\Delta \subseteq C \times \mathsf{Act} \times C$ is the transition relation, and $c_0 \in C$ is the initial state. We write

$$c \xrightarrow{a} c'$$

when the system can move from c to c' by performing action a. The relation need not be deterministic: several transitions with the same source and label may be possible.

A finite run is a sequence

$$c_0 \xrightarrow{a_0} c_1 \xrightarrow{a_1} \cdots \xrightarrow{a_{n-1}} c_n.$$

Its trace is the finite word $a_0 a_1 \cdots a_{n-1}$. Similarly we define infinite runs & their traces. A state is reachable if some finite run leads to it. A deadlock is a state with no outgoing transitions.

Operational models and induced LTSs. An operational model describes what a system stores and which one-step updates are allowed. Its configurations contain all information needed to determine the available next steps. The *induced LTS* has:

- one LTS state for each configuration;
- an edge $c \xrightarrow{a} c'$ for each allowed one-step update from c to c' labelled by a;
- the model's initial configuration as the initial LTS state.

The reachable part of the induced LTS contains just the configurations reachable from the initial one. It can be finite or infinite.

Timed automata. A timed automaton has finitely many locations and finitely many clocks, whose values are nonnegative real numbers. A configuration consists of a location and a clock valuation. A delay transition lets time pass by some $d \geq 0$, provided all location invariants remain true throughout the delay. A discrete transition has a guard and may reset clocks. The induced LTS therefore has both delay edges, written $c \xrightarrow{d} c'$, and discrete action edges, written $c \xrightarrow{a} c'$. An infinite run is *Zeno*[1] if it has infinitely many discrete transitions but the sum of its delays is finite.

[1] Zeno of Elea, famous in Antiquity for his paradoxes; relevant here is the paradox of Achilles and the tortoise, which in modern terms is about the possibility of summing an infinite series to get a finite value.

Finite branching and Kőnig's lemma. An LTS is finitely branching if every configuration has only finitely many outgoing transitions. Unfolding all finite runs from the initial state gives a rooted tree. Kőnig's lemma says that every infinite, finitely branching rooted tree has an infinite branch. Consequently, if a finitely branching LTS has infinitely many reachable configurations, it has an infinite run. This conclusion is about the number of steps; by itself, it says nothing about the elapsed time of a run in a timed system.

A useful graph convention. When discussing treewidth, we forget edge directions and labels: two configurations are adjacent in the underlying undirected graph if there is an LTS transition between them in either direction.

Forward and backward reachability. For a transition relation $\longrightarrow$ and a set of configurations X, define

$$\mathrm{Post}^*(X) = \{c' : c \in X \text{ and } c \longrightarrow^* c'\}, \qquad \mathrm{Pre}^*(X) = \{c : c \longrightarrow^* c' \text{ for some } c' \in X\}.$$

Here $\longrightarrow^*$ allows zero or more steps. Thus $\mathrm{Post}^*(\{c_0\})$ is the reachability set from the initial configuration, while $\mathrm{Pre}^*(F)$ contains configurations from which some target in F can be reached.

Semilinear sets. A *linear* set in $\mathbb{N}^d$ has the form

$$\{b + n_1 p_1 + \cdots + n_k p_k : n_1, \ldots, n_k \in \mathbb{N}\},$$

for fixed vectors $b, p_1, \ldots, p_k \in \mathbb{N}^d$. A *semilinear* set is a finite union of linear sets. For one counter, semilinear subsets of $\mathbb{N}$ are exactly the ultimately periodic sets; equivalently, when counter values n are encoded by 1^n, they are exactly the regular unary languages. For stacks, the analogous notion we will use is regularity of the stack words: a set of pushdown configurations is regular if, for each control state, its stack contents form a regular language.

The right representation depends on the model. Counter values are numbers, whereas pushdown configurations contain words. For a context-free grammar, configurations are sentential forms; one can ask whether their language is regular or context-free, and separately whether its Parikh image is semilinear.

Exercises

Exercise 1. [*Reading a finite LTS*] Consider the LTS with states $C = \{r, g\}$, initial state r, and transitions

$$r \xrightarrow{\mathsf{go}} g, \qquad g \xrightarrow{\mathsf{stop}} r, \qquad g \xrightarrow{\mathsf{wait}} g.$$

(a) Draw the LTS and mark its initial state.

(b) List the reachable states, all deadlocks, and the traces of length at most three.

(c) Add one transition so that the system has a deadlock. Give two different choices and explain the difference between them.

Exercise 2. [*One-counter systems*] A one-counter system has a finite set of control states Q and a counter in $\mathbb{N} = \{0, 1, 2, \ldots\}$. A rule

$$p \xrightarrow{a,d} q$$

changes the control state from p to q, performs action a, and changes the counter by $d \in \mathbb{Z}$. From configuration (p, n) it induces

$$(p, n) \xrightarrow{a} (q, n + d)$$

exactly when $n + d \geq 0$.

Take $Q = \{q\}$, initial configuration $(q, 0)$, and rules

$$q \xrightarrow{\mathsf{inc},+1} q, \qquad q \xrightarrow{\mathsf{dec},-1} q.$$

(a) Draw the reachable part of the induced LTS up to counter value 3. Make clear which edges continue beyond the part shown.

(b) Is $(q, 0)$ a deadlock? Which actions are enabled at (q, n)?

(c) Now allow the counter to range over $\mathbb{Z}$ and remove the non-negativity condition. Draw the part with counter values from -2 to 2. Compare the two systems: give a finite trace possible over $\mathbb{Z}$ but impossible over $\mathbb{N}$.

(d) Is the reachable LTS finite in either version? Explain briefly.

Exercise 3. [*Lossy and gainy counter systems*] Start with a one-counter system as in Exercise 2, with rules $p \xrightarrow{a,d} q$ for $d \in \mathbb{Z}$. Configurations are $(p, n) \in Q \times \mathbb{N}$. Define the following two variants.

- In a *lossy* system, before a rule is applied the counter may lose any number of units, and after the rule it may lose any number of units. Thus $(p, n) \xrightarrow{a} (q, m)$ is allowed if there is some $k \leq n$ such that $k + d \geq 0$ and $m \leq k + d$.
- In a *gainy* system, before and after a rule the counter may gain any number of units. Thus $(p, n) \xrightarrow{a} (q, m)$ is allowed if there is some $k \geq n$ such that $k + d \geq 0$ and $m \geq k + d$.

In both cases, k is the counter value when the ordinary rule is applied. For a set X of configurations, write $\mathrm{Post}^+(X)$ and $\mathrm{Pre}^+(X)$ for the configurations reachable by one or more steps, and retain Post^* and Pre^* for reachability by zero or more steps.

(a) For each variant, prove the appropriate closure properties of $\mathrm{Post}^+(X)$ and $\mathrm{Pre}^+(X)$, separately for each control state. Which sets are downward-closed, and which upward-closed?

(b) Prove that every upward-closed or downward-closed subset of $\mathbb{N}$ is semilinear. Use this and the fact that a finite union of semilinear sets is semilinear to show that if X is semilinear, then both $\mathrm{Post}^*(X)$ and $\mathrm{Pre}^*(X)$ are semilinear for lossy and gainy counter systems. [*Hint*: prove first that $\mathrm{Pre}^*(\{c\})$ and $\mathrm{Post}^*(\{c\})$ are semilinear, and then use the fact that

$$\mathrm{Pre}^*(X) = \bigcup_{c \in X} \mathrm{Pre}^*(\{c\}),$$

and similarly for Post, to deduce the result.]

(c) Apply both semantics to the one-state system with the single rule $q \xrightarrow{\mathsf{dec},-1} q$, starting from $(q, 0)$. Describe the reachable counter values after zero or more steps in each system and explain why the two answers differ.

Exercise 4. [*Pushdown systems*] A pushdown system has a finite set of control states and a stack over a finite alphabet. The rightmost symbol is the top of the stack. For this exercise, let the stack alphabet be $\{\bot, 0, 1\}$, where $\bot$ is a permanent bottom symbol. There is one control state q, the initial stack is $\bot$, and the rules are:

- push a 0 or a 1 above the current top, with action push_0 or push_1;
- pop the top symbol with action pop whenever the top is 0 or 1; $\bot$ is never popped.

For example, pushing 1 from stack $\bot 0$ gives $(q, \bot 0) \xrightarrow{\mathsf{push}_1} (q, \bot 01)$.

(a) Draw the reachable induced LTS through stack height three, labelling push and pop edges. (The height is the number of symbols above $\perp$.)

(b) Describe the underlying undirected graph. What familiar tree does it resemble, and how does the root's degree differ from that of other vertices?

(c) Add a second control state r and an action switch that changes q to r and back without changing the stack. What changes in the first two levels of the induced LTS?

(d) Which information must be included in a configuration? Why would recording only the stack, or only the control state, lose information about future behaviour?

Exercise 5. [*Independent steps and interleaving*] Consider a system with configuration $(x, y) \in \{0, 1\}^2$, initially $(0, 0)$. Action a changes x from 0 to 1 and leaves y unchanged; action b changes y from 0 to 1 and leaves x unchanged. Each action can be performed only once.

(a) Construct the induced LTS. Label every state by its pair (x, y).

(b) Show that both traces ab and ba are possible and lead to the same configuration. Draw the resulting diamond.

(c) Identify the two enabled actions at the initial configuration. What does this example suggest about representing concurrency by an LTS whose runs execute one action at a time?

(d) Change the rules so that b is enabled only after a. Which of the two traces remains possible?

Exercise 6. [*From rules to an induced LTS*] For each model in Exercises 2–4, explicitly identify its configuration space, initial configuration, action labels, and one-step transition relation. Then compare the resulting reachable LTSs:

(a) Which have finitely many reachable configurations?

(b) Which have a branching structure, and which have cycles?

Exercise 7. [*Reachability sets: semilinear or regular?*] In this exercise, a one-counter net has finite control and counter updates by fixed integers, enabled when the resulting counter remains nonnegative. A one-counter automaton additionally has guards testing whether the counter is zero or positive. For both models, identify a configuration (q, n) with its control state and counter value.

(a) For a one-counter net, prove or refute that both $\mathrm{Post}^*(X)$ and $\mathrm{Pre}^*(X)$ are semilinear whenever X is semilinear.

(b) Repeat part 1 for one-counter automata with zero tests. State the corresponding regular-language formulation using the unary encoding 1^n. In both parts, note that a finite initial set is a special case of a semilinear set.

(c) For a pushdown system, take a regular set X of configurations. Show that both $\mathrm{Post}^*(X)$ and $\mathrm{Pre}^*(X)$ are regular sets of configurations. What does this imply for a DPDA, viewed as a pushdown system? Is determinism needed for the reachability-set result?

(d) Let G be a context-free grammar with start variable S and terminal alphabet T. Encode a sentential form over $T \cup V$ by treating each variable as a distinct output symbol. Prove that $\mathrm{Post}^*(\{S\})$ is context-free: for each variable A, add a production $A \to \widehat{A}$, where $\widehat{A}$ is a fresh terminal symbol meaning "leave this variable unexpanded." Deduce that the Parikh image is semilinear. Must this language of sentential forms be regular?

(e) For the same grammar, let $K \subseteq T^*$ be a regular language of terminal target words. Show that the sentential forms in $\mathrm{Pre}^*(K)$ form a regular language over $T \cup V$. [*Hint*: take a DFA for K and, for each grammar variable A, record the pairs of DFA states connected by some word generated from A.]

Exercise 8. [*Same language, different transition systems: BPA and PDA*] Language equivalence compares only which finite words are accepted; it does not require the induced LTSs to have the same structure. Consider the BPA process given by the recursive equation

$$X = 1 + a.X + b.X,$$

where 1 denotes successful termination. Its induced LTS has one configuration X with transitions $X \xrightarrow{a} X$ and $X \xrightarrow{b} X$. Its language consists of the words labelling runs that end successfully.

Now consider the PDA with initial control state q, permanent stack symbol $\bot$, and accepting states q and r. It never changes the stack. From q it has transitions labelled a and b back to q, and an additional transition labelled a from q to r. State r has no outgoing transitions. A PDA accepts a word if some run consumes the whole word and ends in an accepting state.

(a) Write a context-free grammar for the language of the BPA process and determine the language accepted by the PDA. Show that both languages are $\{a, b\}^*$.

(b) Draw the two induced LTSs, including the successful-termination or accepting-state information. Are their transition graphs isomorphic?

(c) Explain why this example does not contradict the equivalence between context-free grammars and PDAs: what information does language equivalence forget?

Exercise 9. [*Challenge: a nonsemilinear Turing-machine reachability set*] For this exercise, use the following explicit convention. A Turing-machine configuration has a finite-control state and a tape encoding. At selected checkpoint configurations, the tape has the form

$$\vdash\ a^n \# b^m\ \dashv,$$

with the head at a fixed location. The checkpoint projection records (n, m). A set of configurations is called semilinear here if its projection to the control state and these symbol-count coordinates is semilinear; in particular, every fixed-control-state slice and every coordinate projection of such a set must be semilinear.

(a) Design a deterministic Turing machine that reaches checkpoint configurations with tape contents $\vdash\ a^n \# b^{n^2}\ \dashv$ for every $n \geq 1$, and enters the checkpoint state only for these canonical configurations. Sketch how it can maintain the unary value n and construct the block of n^2 symbols.

(b) Prove that $\{(n, n^2) : n \geq 1\}$ is not semilinear. Use the checkpoint slice to conclude that the projected reachability set of your machine is not semilinear.

(c) Why is the encoding/projection convention important? Explain why "semilinear" is not a representation-free property of arbitrary sets of Turing-machine configurations.

Exercise 10. [*Optional challenge: treewidth of a pushdown graph*] This challenge uses the following normal form. Let Q be a finite set of control states and Γ a finite stack alphabet containing $\bot$. Every transition replaces the top stack symbol A by a word β of length at most two, so that

$$(p, wA) \longrightarrow (q, w\beta), \qquad |\beta| \leq 2,$$

and no transition removes $\perp$. Consider the underlying undirected graph of all configurations reachable from a fixed initial configuration.

A tree decomposition consists of a tree of bags of vertices such that every vertex and edge occurs in some bag, and the bags containing any fixed vertex form a connected subtree. The width is the largest bag size minus one.

(a) Use the tree of stack words as a decomposition tree. For each word $w \in \Gamma^*$, consider the bag

$$B_w = (Q \times \{w\}) \cup \bigcup_{A \in \Gamma} (Q \times \{wA\}).$$

Intersect each bag with the vertex set of G if G contains only reachable configurations. Explain why every configuration occurs in at least one bag and why the bags containing a fixed configuration are connected.

(b) Check that every possible transition edge has both endpoints in a common bag. Separate the cases where the stack height stays the same, decreases by one, or increases by one.

(c) Deduce the bound

$$\mathrm{tw}(G) \leq |Q|(|\Gamma| + 1) - 1.$$

Exercise 11. [*Timed automata, infinite branching, and Zeno runs*] Consider a timed automaton with one location ℓ, one clock x, and initial configuration $(\ell, x = 0)$. The invariant at ℓ is $x \leq 1$. There is one discrete transition, labelled a, from ℓ to itself, with guard $x > 0$ and reset $x := 0$.

(a) Describe its induced LTS. Specify the configurations and the delay and discrete transitions. Give a schematic picture of the reachable part rather than trying to draw every valuation.

(b) Determine the set of reachable clock valuations. Is the induced LTS finitely branching? Justify your answer.

(c) Give an infinite run that performs a infinitely often but has finite total elapsed time. Verify that every delay respects the invariant and that the guard for a is satisfied each time.

(d) State Kőnig's lemma and use the run tree to explain why, in a finitely branching LTS, infinitely many reachable configurations imply an infinite run. Why does this not imply that every infinite run of a timed automaton has unbounded elapsed time?

(e) Identify the hypothesis of this result that fails for the induced LTS above. Distinguish an infinite run (infinitely many transitions) from a run with unbounded accumulated duration.

Further questions for discussion. For a first meeting, useful habits beyond drawing the graph are to state the configuration precisely, distinguish the full LTS from its reachable part, check when actions are enabled, find deadlocks and cycles, compare traces, and look for simple invariants. When two actions appear independent, check whether both execution orders are possible and reach the same state. These checks apply to finite-state systems, counters, stacks, and token systems alike.